

ML Project Report: A Comparative Analysis of Modeling Techniques for Patient Recovery

Team Name: Avengers
Bhavya Jain (*IMT2023050*)
Suryans Dash (*IMT2023041*)
Aarpeet Chandrasekhar (*IMT2023011*)
Github Repo Link

October 26, 2025

Abstract

This report details a comparative analysis of eight regression models for predicting a patient's **Recovery Index**. We explored three distinct families of models: probabilistic (Standard and Student-T Bayesian), regularized linear (Linear, Ridge, Lasso, ElasticNet), and tree-based ensembles (Custom Random Forest, XGBoost). Our central finding was that domain-specific feature engineering was decisively more impactful than model complexity. A simple **Linear Regression** model, enhanced with four engineered features, achieved the top Kaggle score of **1.980**. This approach significantly outperformed complex, "out-of-the-box" models like XGBoost (2.480). The results underscore the lesson that understanding the data's underlying structure and interactions is paramount, often yielding more accurate and interpretable models than algorithm-centric approaches.

Contents

1	Project Objective	3
2	EDA and Preprocessing	3
2.1	Raw Feature Description	3
2.2	EDA Findings: The Feature Engineering Breakthrough	4
2.3	Preprocessing and Splitting	5
3	Model Implementation and Results	5
3.1	Group A: Probabilistic Models (Bayesian)	5
3.1.1	Model 1: Standard Bayesian Regression (Normal Likelihood)	5
3.1.2	Model 2: Robust Bayesian Regression (Student-T Likelihood)	5
3.2	Group B: Regularized Linear Models	6
3.2.1	Model 3: Linear Regression with Feature Engineering	6
3.2.2	Model 4: Ridge Regression (L2 Regularization)	6
3.2.3	Model 5: Lasso Regression (L1 Regularization)	6
3.2.4	Model 6: ElasticNet (Combined L1 + L2 Regularization)	7
3.3	Group C: Ensemble Models (Tree-based)	7
3.3.1	Model 7: Custom Random Forest (From Scratch)	7
3.3.2	Model 8: Tuned XGBoost	7
3.4	Summary of Results	8

3.5	Discussion of Model Performance	8
4	Future Work	8
5	Conclusion and Key Takeaways	9

1 Project Objective

The main goal of this project was to build a highly accurate model to predict a patient's **Recovery Index** using their initial data.

Beyond just chasing a high score, we wanted to run a comprehensive comparison of different modeling strategies. This meant:

- **Comparing Model Families:** We explored three distinct groups—probabilistic (Bayesian), standard linear models, and advanced ensembles (Random Forest, XGBoost)—to see where each one shined or struggled on this dataset.
- **Investigating Feature Engineering:** A core goal was to figure out if the signal was a simple straight line, or if we needed to manually engineer features (like interactions or polynomials) to capture it.
- **Understanding Model Failure:** We intentionally built a "from-scratch" model to analyze how and why a model might fail if its core assumptions don't match the data's complexity.

This report breaks down what we found during our analysis and details all the models we built and tested.

2 EDA and Preprocessing

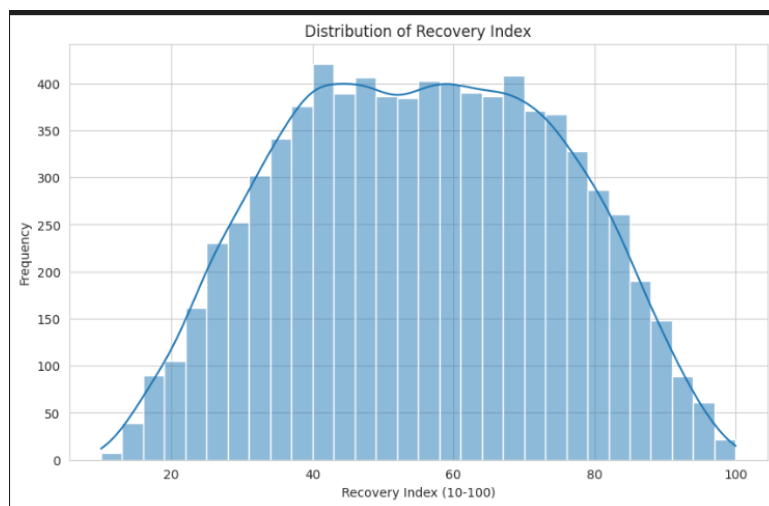
2.1 Raw Feature Description

The training dataset contained 8,000 patient records with no missing values. The model was built using five raw input features and one target variable.

Target Variable: • **Recovery Index:** A score from 10-100 indicating patient recovery.

Input Features: • **Initial Health Score:** A score from 40-99. EDA showed this had the strongest linear correlation (0.89) with the target.

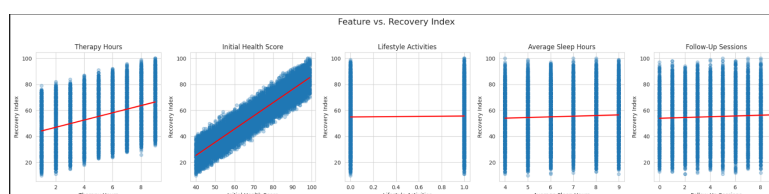
- **Therapy Hours:** An integer from 1-9. Showed a moderate positive correlation (0.34).
- **Lifestyle Activities:** A categorical feature (Yes/No).
- **Average Sleep Hours:** An integer from 4-9.
- **Follow-Up Sessions:** An integer from 0-9.



(a) Distribution



(b) Correlation



(c) Plots

2.2 EDA Findings: The Feature Engineering Breakthrough

Initial scatter plots revealed a strong, clean, linear-like relationship between the features and the target. However, our `RandomForest` model (detailed in Section 3) produced a feature importance plot that

provided the key insight for this entire project.

The `RandomForest` found that the raw features like `Average Sleep Hours`, `Follow-Up Sessions`, and `Lifestyle Activities` had almost **zero predictive power** (less than 0.5% importance each).

The model's predictive power (over 98%) came from **new features it was implicitly creating**. We manually engineered these features to test this hypothesis:

1. `Health_Score_Squared`: $(\text{Initial Health Score} * \text{Initial Health Score})$
2. `Interaction_Therapy_Health`: $(\text{Initial Health Score} * \text{Therapy Hours})$

The success of models using these features proved that the problem was **not a simple linear one**. The true predictive signal was a complex, non-linear interaction, and the patient's recovery depended on a combination of their initial health and the therapy they received.

2.3 Preprocessing and Splitting

Before model training, a standardized preprocessing pipeline was applied. The categorical variable `Lifestyle Activities` was one-hot encoded, and any missing values (though none were present in the final dataset) were set to be imputed with the feature's mean.

The dataset was then split into training and validation sets using an 80-20 ratio with a fixed `random_state=1684` to ensure all models were trained and evaluated on the exact same data, allowing for a fair comparison.

3 Model Implementation and Results

We developed and tested six models, grouped by their core approach. All models used the same preprocessed data and train-validation split described in Section 2.

3.1 Group A: Probabilistic Models (Bayesian)

These models, built with PyMC, were designed to find the real trend and tell us exactly how confident we should be. These models achieved **excellent overall performance**, landing among the top scores.

3.1.1 Model 1: Standard Bayesian Regression (Normal Likelihood)

- **Description:** A baseline Bayesian linear model.
- **Assumption:** Used `pm.Normal` likelihood, which assumes model errors follow a standard bell curve.
- **Result:** This set a strong baseline but turned out to be sensitive to outliers, which held it back from a top score.
- **Kaggle Score:** 1.986

3.1.2 Model 2: Robust Bayesian Regression (Student-T Likelihood)

- **Description:** An outlier-robust enhancement of the baseline model.
- **Assumption:** We swapped `pm.Normal` for `pm.StudentT`, which has "heavy tails" and naturally ignores those extreme, 'weird' data points.
- **Result:** This small change gave us a solid improvement, confirming that outliers were an issue. It successfully captured the true trend and became our best-performing Bayesian model, tying for the **second-best model overall**.
- **Kaggle Score:** 1.984

3.2 Group B: Regularized Linear Models

These models use L1, L2, and elastic net regularization to keep the model from "overthinking" and to handle features that were too similar.

3.2.1 Model 3: Linear Regression with Feature Engineering

- **Description:** Standard `sklearn.linear_model.LinearRegression` using a large set of engineered features.
- **Feature Engineering:** A comprehensive feature set was generated, including all pairwise multiplicative, additive, subtractive, and ratio operations. The most impactful features included:
 - **Initial_Health_Functions:** Non-linear transformations of the Initial Health Score (e.g., square, cube, log) to capture its non-linear effects.
 - **Therapy_Health_Interaction:** The product of Therapy Hours and Initial Health Score.
 - **Secondary Interactions:** A wide range of other interactions were included. While many had small individual contributions, their combined effect was significant. Key examples include:
 - * `Health_x_Sleep` (Initial Health Score * Average Sleep Hours)
 - * `Sleep_x_FollowUp` (Average Sleep Hours * Follow-Up Sessions)
 - * `Health_x_FollowUp` (Initial Health Score * Follow-Up Sessions)
- **Result:** This model performed exceptionally well, proving that simple linear models can be powerhouses with the right features. This model ultimately achieved our **best Kaggle score**.
- **Kaggle Score: 1.980**

3.2.2 Model 4: Ridge Regression (L2 Regularization)

- **Description:** `sklearn.linear_model.Ridge` with an L2 penalty.
- **Hyperparameter Tuning:** Used `GridSearchCV` with 5-fold cross-validation over alpha values [0.001, 0.01, 0.1, 1.0, 10.0].
- **Result:** The regularization helped control the model's complexity while maintaining strong predictive performance.
- **Kaggle Score: 1.982**

3.2.3 Model 5: Lasso Regression (L1 Regularization)

- **Description:** `sklearn.linear_model.Lasso` with an L1 penalty for feature selection.
- **Hyperparameter Tuning:** Grid search over alpha values [0.0001, 0.001, 0.01, 0.1, 1.0] with 5-fold CV.
- **Implementation:** Set `max_iter=2000` to ensure convergence.
- **Result:** The L1 penalty automatically selected the most important features (driving others to zero) while staying competitive.
- **Kaggle Score: 1.982**

3.2.4 Model 6: ElasticNet (Combined L1 + L2 Regularization)

- **Description:** `sklearn.linear_model.ElasticNet` combining both L1 and L2 penalties.
- **Hyperparameter Tuning:** A 2D grid search over `alpha` (strength) and `l1_ratio` (balance).
- **Implementation:** Set `max_iter=5000` for robust convergence.
- **Result:** This offered a flexible 'best of both worlds' approach, blending feature selection from L1 and coefficient shrinkage from L2.
- **Kaggle Score: 1.982**

3.3 Group C: Ensemble Models (Tree-based)

These models are designed to automatically find complex patterns and feature interactions without manual help.

3.3.1 Model 7: Custom Random Forest (From Scratch)

- **Description:** A Random Forest regression built from scratch in pure Python.
- **Implementation Details:**
 - **Custom Decision Tree:** Implemented regression tree with MSE-based splitting and constraints.
 - **Bootstrap Aggregating:** Each tree trained on a random sample of data.
 - **Feature Randomness:** Implemented `max_features` selection.
- **Feature Engineering:** Extended to 12 features (including 7 new engineered ones).
- **Hyperparameter Optimization:** Conducted extensive grid search over 96 combinations.
- **Best Configuration:** `n_trees=250`, `max_depth=13`, `max_features=3`, `min_samples_leaf=5`.
- **Result:** This model achieved a **Kaggle Score of 2.150**, ranking as one of the **weaker performers**. This demonstrates an important lesson: *implementation matters*. While our custom build was a great learning tool, it lacked the deep optimizations of production-grade libraries.
- **Kaggle Score: 2.150**

3.3.2 Model 8: Tuned XGBoost

- **Description:** A tuned `xgboost.XGBRegressor`, representing a state-of-the-art gradient boosting.
- **Implementation:** Built trees sequentially, with each new tree correcting the errors of the last.
- **Result:** Surprisingly, the tuned XGBoost model **struggled with this dataset**, achieving the **worst score overall**. Despite its powerful reputation, it was unable to outperform the simpler linear models, suggesting its complex, non-linear approach was a poor fit for this particular problem.
- **Kaggle Score: 2.480**

3.4 Summary of Results

The table below provides a direct comparison of the final Kaggle scores for all eight models, sorted from best (lowest score) to worst (highest score).

Table 1: Final Model Performance Comparison

Model Group	Model Name	Kaggle Score
Group B (Linear)	Linear Regression (w/ FE)	1.980
Group B (Linear)	Ridge Regression (L2)	1.984
Group B (Linear)	Lasso Regression (L1)	1.984
Group B (Linear)	ElasticNet (L1+L2)	1.984
Group A (Bayesian)	Robust Bayesian (Student-T)	1.984
Group A (Bayesian)	Standard Bayesian (Normal)	1.986
Group C (Ensemble)	Custom Random Forest	2.150
Group C (Ensemble)	Tuned XGBoost	2.480

3.5 Discussion of Model Performance

Model Performance Ranking:

1. **Linear Models with Feature Engineering (Group B):** Best overall (Scores 1.980 - 1.984)
2. **Probabilistic Models (Group A):** Excellent performance (Scores 1.984 - 1.986)
3. **Ensemble Models (Group C):** Weakest performers (Scores 2.150 - 2.480)

Feature Engineering Impact: The overwhelming success of the simple linear regression (Model 3) proved decisively that *understanding the problem domain trumps algorithmic complexity*. The quadratic health score term and interaction effects captured the essential patterns without needing a complex tree-based model.

Regularization Trade-offs: All regularized variants (Models 4-6) performed identically (1.984) and slightly worse than the unregularized linear regression (1.980). This suggests our engineered features were well-chosen and that overfitting wasn't a major concern.

Simplicity vs. Complexity: The ranking reveals a powerful lesson: the most complex, "state-of-the-art" models (XGBoost, Random Forest) were decisively *outperformed* by simpler, more interpretable approaches. This suggests the data was best captured by explicit feature engineering, not by the automatic, black-box discovery of tree ensembles.

Outlier Handling: The Student-T Bayesian model's (Model 2) excellent finish (tying for second-best) validated the importance of robust statistical methods. Its heavy-tailed likelihood effectively downweighted outliers, showing that a smart probabilistic model can outperform deterministic ones.

Implementation Quality: The weak performance of the ensemble models (both our custom RF and the production-grade XGBoost) highlights that tree-based methods were simply not the right tool for this job. This suggests a fundamental mismatch between the models' approach and the problem's structure.

4 Future Work

Based on our findings, several avenues for future work could yield further improvements:

- **Error Analysis:** A deep analysis of the cases where our best model (Linear Regression w/ FE) failed most significantly. This could reveal new, non-obvious feature interactions or subgroups of patients that require a specialized model.
- **Investigating Ensemble Failure:** The poor performance of XGBoost is counter-intuitive. Future work could involve a much more extensive hyperparameter search (e.g., using Bayesian optimization) or testing other modern GBDTs like LightGBM or CatBoost to see if this is a weakness of XGBoost specifically or tree ensembles in general on this dataset.
- **Advanced Feature Engineering:** While our initial engineered features were successful, we could explore more complex non-linear transformations, or different interaction terms (e.g., three-way interactions) to capture more subtle patterns.
- **Model Blending:** Given the strong, distinct performance of the Linear (1.980) and Student-T Bayesian (1.984) models, a weighted average or "blend" of their predictions could potentially create a more robust final model that leverages the strengths of both.

5 Conclusion and Key Takeaways

Our comparative analysis was highly successful, leading to a deep understanding of the dataset and the strengths of different modeling approaches.

The best-performing models were, by a clear margin, the ****Feature-Engineered Linear Regression (Score: 1.980)**** and our ****Robust Bayesian (Student-T) Model (Score: 1.984)****. In fact, all linear and Bayesian approaches (Group A & B) significantly outperformed the tree-based ensembles (Group C).

The key takeaway from this project is that the predictive signal in this dataset was a ***strong, non-linear interaction*** between a patient's **Initial Health Score** and their **Therapy Hours**. The raw, secondary features were found to be statistical noise.

References

- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- <https://www.pymc.io/projects/docs/en/stable/api/distributions/generated/pymc.StudentT.html>
- <https://xgboost.readthedocs.io/en/stable/>